

ECE 695: Introduction to Operating Systems

Lab 1

System Stack

1 The C Model

Modern Operating Systems are written mainly in some high-level language such as C/C++. The memory layout of a C program is as follows:

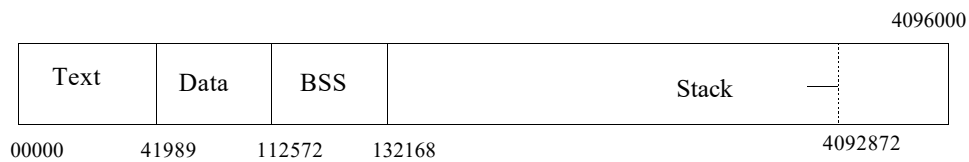


Figure 1: Memory layout of a C program

- text segment: contains the instructions/code
- data segment: contains initialized (hard-coded) data
- BSS segment: contains uninitialized data (global variables/malloc). Also known as heap.
- Stack: local storage for various subroutines

2 Stack

2.1 The General Concept

- Stack segment grows dynamically (see Fig. 2).
- Stack is used to
 1. store local variables
 2. pass function parameters
 3. save registers across function calls
- Memory space on the stack that is currently being used is called the “stack frame.”
- A call to a subroutine results in creation of a new stack frame.

- Stack frame is accessed using the “stack-frame pointer,” typically stored in one of the processor registers. Intel and many other architectures provide a special register called SP for this purpose. Other architectures leave it to the OS to decide which register to use as the stack-frame pointer. In dlxos, we use the register r29 to point to the top of the stack.

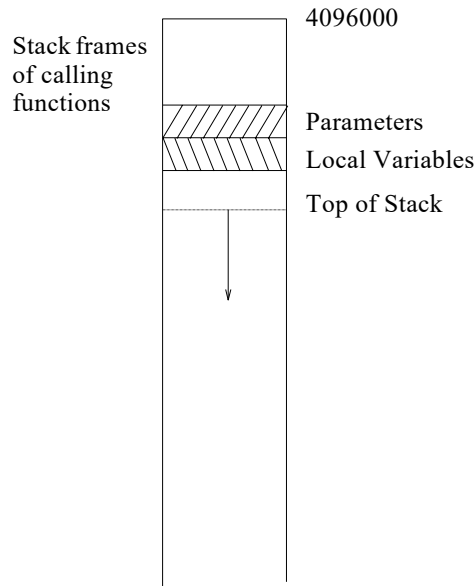


Figure 2: Typical stack for a program

2.2 User Stack

- User stack is used by user process for function calls when running in user mode
- It is used to store local variables, function parameters, and registers being destroyed
- Functions’ memory requirements change depending on number of parameters, local variables, etc.
- Stack-frame size is not fixed, but is typically determined by the compiler

2.3 System Stack

- Process context consists of
 - General purpose registers (r1 through r31)
 - Floating point registers (f0 through f30)

– special registers

- In DLXOS, system stack is used to store the process context when a system call is invoked
- It is also used to pass parameters to system functions, and maybe some local area for computation
- System stack is allocated in OS space, and hence needs to be allocated by the OS
- Every process is allocated separate system stack
- OS cannot determine the memory requirements of a routine in a straight-forward fashion, hence allocates fixed-sized stack frames. Thus, OS routines need to be written more carefully
- Some OS saves most of the process context in the PCB

3 The C Convention

In C, all the local variables are stored on the stack. So are the registers that are going to be destroyed, and the function parameters (note that as C uses call by value, function parameters are nothing but local variables). ANSI C standard requires that a C compiler pass the function parameters from right to left on the stack.

The DLXOS C compiler follows the ANSI convention while pushing parameters on the stack. As already mentioned, in dlxos, we use the register r29 to store the stack-frame pointer. Thus 0(r29) is the leftmost function parameter, 4(r29) is the second function parameter, and so on. We need to know these conventions, because we will be interfacing assembly code with the C code.

Thus, a function call in C is translated into following three steps in assembly language:

1. Save the processor registers that are going to be destroyed onto the system stack
2. Save the function parameters on the top
3. jump to the subroutine address

To call a function from the assembly language, we have to follow these three steps “manually.”